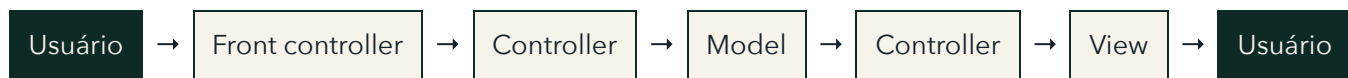


Manual: banco, cadastro e gerência de usuários

Três tarefas, no padrão MVC. Não misturem SQL na view nem HTML no model.

Tarefas

1. Nova definição do banco
2. Reconfigurar o SQLite na próxima execução
3. Arrumar o registro do usuário
4. Criar a gerência de usuários



Copiem o dashboard: `DashboardController` pede dados ao model, recebe o resultado e só então chama a view. Modelo completo no PDF [documentacao/mvc/guia-mvc.pdf](#).

TAREFA 1

Onde colocar a definição do banco

A definição das tabelas **não vai em PHP**. Vai em SQL, neste arquivo:

`/Users/marcelo/Documents/desenvolvimento/mahjong/database/schema.sql`

Esse caminho está em `config.php` (`schema_path`). O programa lê o arquivo em `src/db.php` e executa o SQL na abertura da conexão.

Arquivo	O que vocês fazem
<code>database/schema.sql</code>	Substituem (ou completam) pelo novo schema da aula: tabelas, colunas, chaves. É o arquivo principal desta tarefa.
<code>database/seed.sql</code>	Ajustem o INSERT do admin se as colunas de <code>users</code> (ou outras) mudarem. Hoje o seed cria o usuário <code>admin</code> / senha <code>admin</code> .

`config.php`

Só mexam se o professor pedir outro nome de arquivo. Não criem um segundo schema “solto” fora desses caminhos: o PHP não vai lê-lo.

O programa guarda uma impressão digital (hash) de `schema.sql` na tabela `_schema_meta`. Se o arquivo SQL mudar, na próxima execução o SQLite é recriado sozinho. Vocês **não precisam apagar** `data/app.sqlite` na mão. Uma cópia antiga fica em `data/app.sqlite.bak`.

TAREFA 2

Como o banco é reconfigurado na próxima execução

1. Salvem o novo conteúdo em `database/schema.sql` (e o seed, se as colunas do admin mudarem).
2. Abram de novo o site em `http://localhost:8081` (se o servidor já estiver no ar, basta recarregar a página).
3. `src/db.php` compara o hash do `schema.sql` com o gravado no banco. Se for diferente, recria `data/app.sqlite`, aplica o schema e o seed.

Recriar o banco **zera os dados de teste** (cadastros que vocês tiverem feito). O admin volta a ser o do `seed.sql`. Não apaguem `database/schema.sql` nem `database/seed.sql`.

Não existe atualização tabela a tabela. Ou o `schema.sql` é o mesmo (hash igual) e o banco fica como está, ou o arquivo mudou e o SQLite inteiro é recriado com todas as tabelas novas.

TAREFA 3

Arrumar o registro do usuário

O cadastro público já existe (`/cadastro`), mas ainda está no schema antigo: nome, e-mail e senha (mínimo 8 caracteres; o login do admin usa o “e-mail” `admin`). Depois do novo banco, o formulário e o `INSERT` precisam bater com as colunas novas.

Fluxo atual (não criem outra rota se esta continuar valendo):

`GET/POST /cadastro` → `AuthController` → `User::register()` → view `views/auth/register.php`

Arquivos para alterar

Arquivo	Por quê
views/auth/register.php	View. Incluir os campos novos do schema (usuário, telefone, etc.). Manter <code>csrfField()</code> . Sem SQL aqui.
src/controllers/AuthController.php	Controller. Em <code>showRegister()</code> e <code>register()</code> , ler os novos campos com <code>request()</code> e passar para o model / devolver à view em caso de erro.
src/models/User.php	Model. Ajustar <code>register()</code> , <code>create()</code> , <code>emailExists()</code> / busca por login. Validar e gravar as colunas novas. SQL só aqui.
src/AuthService.php	Se o login deixar de ser o campo <code>email</code> , alinhar <code>attempt()</code> com o que o model busca (usuário, e-mail, etc.).
views/auth/login.php	Se o identificador mudar (ex.: usuário em vez de e-mail), o formulário de entrar precisa do mesmo campo.
database/seed.sql	Admin inicial tem de inserir todas as colunas NOT NULL do novo <code>users</code> .

Arquivos que provavelmente não precisam ser criados

Não criem um segundo `RegisterController` se o cadastro continuar em `/cadastro` . A rota já está em `public/index.php` :

```
$router->get('/cadastro', [$auth, 'showRegister']);
$router->post('/cadastro', [$auth, 'register']);
```

Se o schema tiver tabela `members` ligada a `users` , o cadastro pode criar o usuário e o membro no model (ou chamar `Member::create()`). A view não faz o INSERT.

TAREFA 4

Criar a gerência de usuários

Hoje o menu **Usuários** (`/usuarios`) cai no `PageController::usuarios()` e abre só o placeholder `views/coming-soon.php` . Vocês devem promover isso a um módulo de verdade, no mesmo desenho do dashboard.

Arquivos para criar

Arquivo novo	Camada	Papel
<code>src/controllers/UserController.php</code>	Controller	Listar, criar (se for no admin), editar, ativar/desativar. <code>requireAuth()</code> ; o ideal é restringir a <code>role = admin</code> .
<code>views/users/index.php</code>	View	Tabela de usuários (como a captura de eventos: nome, papel, status, ações).
<code>views/users/form.php</code> (se precisarem)	View	Formulário de novo/editar. Campos iguais aos do schema.

Arquivos para alterar

Arquivo existente	O que mudar
<code>public/index.php</code>	Front controller. Instanciar <code>UserController</code> e trocar a rota: <code>\$router->get('/usuarios', [\$users, 'index']);</code> Acrescentar POST de criar/editar/excluir conforme a tela (ex.: <code>/usuarios/novo</code> , <code>/usuarios/salvar</code>).
<code>src/controllers/PageController.php</code>	Remover o método <code>usuarios()</code> quando a rota nova estiver no ar. Senão ficam dois donos para a mesma URL.
<code>src/models/User.php</code>	Métodos de gerência: <code>all()</code> , <code>findById()</code> , <code>update()</code> , etc. O controller não escreve SQL cru.
<code>views/layout.php</code>	O link do menu já aponta para <code>/usuarios</code> . Só mudem se a URL da gerência for outra.
<code>public/assets/css/app.css</code>	Estilo da tabela/formulário. Nada de <code>style="..."</code> no PHP.

Ordem sugerida na aula

1. Novo schema + recarregar o site (o programa recria o SQLite sozinho).
2. Ajustar cadastro público (view → controller → model).
3. Criar `UserController` + view de lista + métodos no model + rota no `index.php` .

Não implementem a gerência só na view, nem só no `PageController` . Placeholder sai; módulo novo entra com controller próprio, como o dashboard.

Clube de Mahjong • manual da aula • <documentacao/aula/manual.php>